HOA SEN UNIVERSITY

www.hoasen.edu.vn

Introduction PHP

Basic PHP Syntax

What is PHP?

- PHP == 'Hypertext Preprocessor'
- Open-source, server-side scripting language
- Used to generate dynamic web-pages
- PHP scripts reside between reserved PHP tags
  - This allows the programmer to embed PHP scripts within HTML pages

What is PHP (cont'd)

- Interpreted language, scripts are parsed at run-time rather than compiled beforehand
- Executed on the server-side
- Source-code not visible by client
  - 'View Source' in browsers does not display the PHP code
- Various built-in functions allow for fast development
- Compatible with many popular databases

What does PHP code look like?

- Structurally similar to C/C++
- Supports procedural and object-oriented paradigm (to some degree)
- All PHP statements end with a semi-colon
- Each PHP script must be enclosed in the reserved PHP tag

```
<?php
    ...
?>
```

Comments in PHP

- Standard C, C++, and shell comment symbols

```
// C++ and Java-style comment

# Shell-style comments

/* C-style comments
     These can span multiple lines */
```

Variables in PHP

- PHP variables must begin with a “$” sign
- Case-sensitive ($Foo != $foo != $fOo)
- Global and locally-scoped variables
  - Global variables can be used anywhere
  - Local variables restricted to a function or class
- Certain variable names reserved by PHP
  - Form variables ($_POST, $_GET)
  - Server variables ($_SERVER)
  - Etc.

Variable usage

```
<?php
$foo = 25;            // Numerical variable
$bar = "Hello";       // String variable

$foo = ($foo * 7);    // Multiplies foo by 7
$bar = ($bar * 7);    // Invalid expression
?>
```

Echo

- The PHP command ‘echo’ is used to output the parameters passed to it
  - The typical usage for this is to send data to the client’s web-browser
- Syntax
  - void echo (string arg1 [, string argn...])
  - In practice, arguments are not passed in parentheses since echo is a language construct rather than an actual function

Echo example

```
<?php
$foo = 25;              // Numerical variable
$bar = "Hello";         // String variable

echo $bar;              // Outputs Hello
echo $foo,$bar;         // Outputs 25Hello
echo "5x5=",$foo;       // Outputs 5x5=25
echo "5x5=$foo";        // Outputs 5x5=25
echo '5x5=$foo';        // Outputs 5x5=$foo
?>
```

- Notice how echo '5x5=$foo' outputs $foo rather than replacing it with 25
- Strings in single quotes (' ') are not interpreted or evaluated by PHP
- This is true for both variables and character escape-sequences (such as "\n" or "\\")

Arithmetic Operations

```
<?php
        $a=15;
        $b=30;
        $total=$a+$b;
        Print $total;
        Print "<p><h1>$total</h1>";
        // total is 45
?>
```

- $a - $b // subtraction
- $a * $b // multiplication
- $a / $b // division
- $a += 5 // $a = $a+5 Also works for *= and /=

Concatenation

- Use a period to join strings into one.

```
<?php
$string1="Hello";
$string2="PHP";
$string3=$string1 . " " . $string2;
Print $string3;
?>
```

```
Hello PHP
```

Escaping the Character

- If the string has a set of double quotation marks that must remain visible, use the \ [backslash] before the quotation marks to ignore and display them.

```
<?php
$heading="\"Computer Science\"";
Print $heading;
?>
```

```
"Computer Science"
```

PHP Control Structures

- Control Structures: Are the structures within a language that allow us to control the flow of execution through a program or script.
- Grouped into conditional (branching) structures (e.g. if/else) and repetition structures (e.g. while loops).
- Example if/else if/else statement:

```
if ($foo == 0) {
      echo 'The variable foo is equal to 0';
}
else if (($foo > 0) && ($foo <= 5)) {
       echo 'The variable foo is between 1 and
5';
}
else {
       echo 'The variable foo is equal to '.
$foo;
}
```

If … Else…

- if (condition)
  {
  Statements;
  }
  else
  {
  Statement;
  }

```
<?php
if($user=="John")
{
        print "Hello John.";
}
else
{
        print "You are not John.";
}
?>
```

No THEN in PHP

While Loops

- While (condition)
  {
  Statements;
  }

```
<?php
$count=0;
While($count<3)
{
    Print "hello PHP. ";
    $count += 1;
    // $count = $count + 1;
    // or
    // $count++;
?>
```

```
hello PHP. hello PHP. hello PHP.
```

Arrays

- An array in PHP is actually an ordered map which maps values to keys.
- An array can be thought of in many ways:

- Linearly indexed array , list (vector), hash table (which is an implementation of a map), dictionary, collection, stack (LIFO), queue (FIFO)

Arrays

- Kinds of arrays:
  - numeric arrays.
  - associative arrays.
  - multi dimensional arrays.

Arrays

- In numeric arrays each key value corresponds to numeric values.
- They can be divided into two categories

  1.automatic numeric array index.

  2.manual array numeric index.

<u>automatic numeric array index</u>

```
<?php
   $x=array(1,2,3);
   print_r($x);
?>
```

Out put: Array ([0] => 1 [1] => 2 [2] => 3)

Arrays

```
$name = array();          # create
$name = array(value0, value1, ..., valueN);
$name[index]              # get element value
$name[index] = value;     # set element value
$name[] = value;          # append                    PHP
```

```
$a = array();       # empty array (length 0)
$a[0] = 23;         # stores 23 at index 0 (length 1)
$a2 = array("some", "strings", "in", "an", "array");
$a2[] = "Ooh!";     # add string to end (at index 5)
```

- Append: use bracket notation without specifying an index
- Element type is not specified; can mix types

Arrays

- Manual array numeric index

```
<?php
 $x[2]=10;   $x[3]=50;//$x=array(2=>10,3=>50);
 echo $x[2];   echo $x[3];
?>
```

- Associative arrays

In associated arrays each ID associated with its value

```
<?php
    $x=array("ab"=>1,"cd"=>2,"xy"=>3);
    print_r($x);
?>
```

Arrays

- Multidimensional Arrays-An array contains one or more arrays

```
<?php
   $z=array(array(10,20,30),array(40,50,60));
   print_r($z);
?>
```

Array ([0] => Array ([0] => 10 [1] => 20 [2] => 30)
[1] => Array ([0] => 40 [1] => 50 [2] => 60))

Arrays

```
<?php
      $x=array("ab"=>1,array(2,3,4),"cd"=>8);
      print_r($x);
?>
```

Array (["ab"] => 1 [0] => Array ([0] => 2 [1] => 3 [2] => 4) ["cd"] => 8)

Arrays

```
<?php
      $x=array(3=>4,array(2,3,4),5);
      print_r($x);
?>
```

Array ([3] => 4 [4] => Array ([0] => 2 [1] => 3 [2] => 4) [5] => 5)

Array functions

function name(s)	description
count	number of elements in the array
print_r	print array's contents
array_pop, array_push, array_shift, array_unshift	using array as a stack/queue
in_array, array_search, array_reverse, sort, rsort, shuffle	searching and reordering
array_fill, array_merge, array_intersect, array_diff, array_slice, range	creating, filling, filtering
array_sum, array_product, array_unique, array_filter, array_reduce	processing elements

Array function example

- The array in PHP replaces many other collections in Java
  - list, stack, queue, set, map, ...

```
$tas = array("MD", "BH", "KK", "HM", "JP");
for ($i = 0; $i < count($tas); $i++) {
      $tas[$i] = strtolower($tas[$i]);
}
$morgan = array_shift($tas);
array_pop($tas);
array_push($tas, "ms");
array_reverse($tas);
sort($tas);
$best = array_slice($tas, 1, 2);
```

Output:

```
# ("md", "bh", "kk", "hm", "jp")
# ("bh", "kk", "hm")
# ("bh", "kk", "hm", "ms")
# ("ms", "hm", "kk", "bh")
# ("bh", "hm", "kk", "ms")
# ("hm", "kk")
```

foreach loop

```
foreach ($array as $variableName) {
        ...
}
```
PHP

```
$fellowship = array("Frodo", "Sam", "Gandalf",
"Strider", "Gimli", "Legolas", "Boromir");

print "The fellowship of the ring members are: \n";

for ($i = 0; $i < count($fellowship); $i++) {
        print "{$fellowship[$i]}\n";
}
print "The fellowship of the ring members are: \n";

foreach ($fellowship as $fellow) {
        print "$fellow\n";
}
```
PHP

Multidimensional Arrays

```
<?php $AmazonProducts = array(
            array("BOOK", "Books", 50),
            array("DVDs", "Movies", 15),
            array("CDs", "Music", 20)
   );
for ($row = 0; $row < 3; $row++) {
      for ($column = 0; $column < 3; $column++) {
      ?>
      <p> | <?= $AmazonProducts[$row][$column] ?>
      <?php } ?>
      </p>
<?php } ?>
```

PHP

Multidimensional Arrays (cont.)

```
<?php $AmazonProducts = array(
    array("Code" =>"BOOK", "Description" => "Books", "Price" => 50),
    array("Code" => "DVDs", "Description" => "Movies", "Price" => 15),
    array("Code" => "CDs", "Description" => "Music", "Price" => 20)
);

for ($row = 0; $row < 3; $row++) { ?>
    <p> | <?= $AmazonProducts[$row]["Code"] ?> |
        <?= $AmazonProducts[$row]["Description"] ?> |
        <?= $AmazonProducts[$row]["Price"] ?>
    </p>
<?php } ?>
```

PHP

String compare functions

Name	Function
strcmp	compareTo
strstr, strchr	find string/char within a string
strpos	find numerical position of string
str_replace, substr_replace	replace string

- Comparison can be:
  - Partial matches
  - Others
- Variations with non case sensitive functions
  - strcasecmp

String compare functions examples

```
str_replace ( mixed $search , mixed $replace , mixed $subject [, int &$count ] ) : mixed
```

This function returns a string or an array with all occurrences of **search** in **subject** replaced with the given **replace** value.

```php
$test = "Hello World! \n";
print strpos($test, "o");
print strpos($test, "o", 5);
```

PHP

```php
$email  = 'name@example.com';
$domain = strstr($email, '@');
echo $domain; // prints @example.com

$user = strstr($email, '@', true); // As of PHP 5.3.0
echo $user; // prints name
```

PHP

Regular expressions

- Regular expression: a pattern in a piece of text

```
[a-z]at                 #cat, rat, bat…
[aeiou]
[a-zA-Z]
[^a-z]                  #not a-z
[[:alnum:]]+            #at least one alphanumeric char
(very) *large           #large, very very very large…
(very){1, 3}                  #counting "very" up to 3
^bob                    #bob at the beginning
com$                    #com at the end
PHPRegExp
```

EMBEDDED PHP

Printing HTML tags in PHP = bad style

```
<?php
print "<!DOCTYPE html PUBLIC \"-//W3C//DTD XHTML 1.1//
EN\"\n";
print " \"http://www.w3.org/TR/xhtml11/DTD/
xhtml11.dtd\">\n";
print "<html xmlns=\"http://www.w3.org/1999/xhtml\">\n";
print " <head>\n";
print " <title>Geneva's web page</title>\n";
...
for ($i = 1; $i <= 10; $i++) {
print "<p> I can count to $i! </p>\n";
}
?>
```

HTML

- best PHP style is to minimize print/echo statements in embedded PHP code
- but without print, how do we insert dynamic content into the page?

PHP expression blocks

```
<?= expression ?>
```
PHP

```
<h2> The answer is <?= 6 * 7 ?> </h2>
```
PHP

The answer is 42

output

- PHP expression block: a small piece of PHP that evaluates and embeds an expression's value into HTML
  - `<?= expression ?>` is equivalent to:

```
<?php print expression; ?>
```
PHP

Expression block example

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.1//EN"
"http://www.w3.org/TR/xhtml11/DTD/xhtml11.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head><title>CSE 190 M: Embedded PHP</title></head>
<body>
<?php
for ($i = 99; $i >= 1; $i--) {
?>
<p> <?= $i ?> bottles of beer on the wall, <br />
<?= $i ?> bottles of beer. <br />
Take one down, pass it around, <br />
<?= $i - 1 ?> bottles of beer on the wall. </p>
<?php
}
?>
</body>
</html>
```

Common errors: unclosed braces, missing = sign

```
...
<body>
<p>Watch how high I can count:
<?php
for ($i = 1; $i <= 10; $i++) {
?>
      <? $i ?>
</p>
</body>
</html>
```

PHP

- if you forget to close your braces, you'll see an error about 'unexpected `$end`'
- if you forget = in `<?=`, the expression does not produce any output

Complex expression blocks

```
...
<body>
<?php
for ($i = 1; $i <= 3; $i++) {
      ?>
      <h<?= $i ?>>This is a level <?= $i ?>
heading.</h<?= $i ?>>
      <?php
}
?>
</body>
```

PHP

This is a level 1 heading.

This is a level 2 heading.

This is a level 3 heading.

output

Functions

- Functions MUST be defined before then can be called
- Function headers are of the format

```
function functionName($arg_1, $arg_2, ..., $arg_n)
```

- parameter types and return types are not written
- a function with no return statements implicitly returns NULL
- Unlike variables, function names are not case sensitive (foo(…) == Foo(…) == FoO(…))

Functions example

```
<?php
    // This is a function
   function foo($arg_1, $arg_2)
    {
      $arg_2 = $arg_1 * $arg_2;
      return $arg_2;
   }

   $result_1 = foo(12, 3); // Store the
   function
   echo $result_1;         // Outputs 36
   echo foo(12, 3);        // Outputs 36
?>
```

Date Display

```
$datedisplay=date("yyyy/m/d");
print $datedisplay;
# If the date is April 1st,
  2009
# It would display as 2009/4/1
```

2009/4/1

```
$datedisplay=date("l, F m, Y");
Print $datedisplay;
# If the date is April 1st, 2009
# Wednesday, April 1, 2009
```

Wednesday, April 1, 2009

Month, Day & Date Format Symbols

M	Jan
F	January
m	01
n	1

Day of Month	d	01
Day of Month	J	1
Day of Week	I	Monday
Day of Week	D	Mon

Include Files

- `include "header.php";`
- `include ("footer.php");`

- This inserts files; the code in files will be inserted into current code.
- **`require`** is identical to **`include`** except upon failure it will also produce a fatal **`E_COMPILE_ERROR`** level error. In other words, it will halt the script whereas **`include`** only emits a warning (**`E_WARNING`**) which allows the script to continue.

Include Files

- The **`include_once`** statement includes and evaluates the specified file during the execution of the script.
- This is a behavior similar to the **`include`** statement, with the only difference being that if the code from a file has already been included, it will not be included again.
- The **`require_once`** statement is identical to **`require`** except PHP will check if the file has already been included, and if so, not include (require) it again

Q&A